Ingenieria de software 1

Jaime Nazar Anchorena

Contents

Capitulo 1: Software y la ingenieria de software **4**
Naturaleza del software . . . 4
Definicion . . . 4
Caracteristicas que difieren del hardware . . . 4
Categorias . . . 4
Software heredado . . . 4
Naturaleza unica de las webapps . . . 5
Ingenieria de software . . . 5
Definicion . . . 5
Proceso del software . . . 6
Practica de la ingenieria de software . . . 6
Escencia de la practica . . . 6
Principios generales . . . 7

Capitulo 2: Modelos del proceso **7**
Modelo general de proceso . . . 7
Definicion: Actividad estructural . . . 7
Identificacion de un conjunto de tareas . . . 9
Patrones del proceso . . . 9
Evaluacion y mejora del proyecto . . . 9
Modelos del proceso prescriptivo . . . 10
Modelo de procesos especializado . . . 10
Desarrollo basado en componentes . . . 10
Modelo de metodos formales . . . 10
Desarrollo orientado a aspectos . . . 11
El proceso unificado . . . 11
Fases del proceso unificado . . . 11
Modelos del proceso personal y del equipo . . . 11
Proceso personal del software(PPS) . . . 13
Proceso del equipo de software(PES) . . . 13
Tecnologia del proceso . . . 13
TL;DR . . . 14

Capitulo 24: Conceptos de administracion de proyecto **14**

El espectro administrativo . . . 14
El personal . . . 15
Los participantes . . . 15
Lideres de equipo . . . 15
Equipo de software . . . 15
Equipos agiles . . . 17
Conflictos de coordinacio y comunicacion . . . 17
El producto . . . 17
Ambito del software . . . 17
Descomposicion del problema . . . 17
El proceso . . . 17
Fusion de producto y proceso . . . 18
Descomposicion del proceso . . . 18
El proyecto . . . 18
Principio W^5HH . . . 19
Practicas cruciales . . . 19
TL;DR . . . 19

Capitulo 3: Desarrollo agil **19**
Manifesto agil . . . 19
Que es la agilidad? . . . 20
La agilidad y el costo del cambio . . . 20
Que es un proceso agil? . . . 21
Principios de agilidad . . . 21
La politica de desarrollo agil . . . 21
Factores humanos . . . 22
Programacion extrema(XP) . . . 22
Valores . . . 22
Proceso . . . 22
XP industrial . . . 22
Debate XP . . . 24
Otros modelos agiles de proceso . . . 24
TL:DR . . . 24

Capitulo 5: Comprension de los requerimientos **25**
Tareas . . . 25
Concepcion . . . 25
Indagacion . . . 25
Elaboracion . . . 25
Negociacion . . . 26
Especificacion . . . 26
Validacion . . . 26
Administracion de los requerimientos . . . 26
Establecer las bases . . . 26
Indagacion de los requerimientos . . . 27
Recabacion en forma colaborativa . . . 27

Despliegue de la funcion de calidad . . . 27
Escenarios de uso . . . 27
Indagacion(Analisis) de los productos de trabajo . . . 27
Desarrollo casos de uso . . . 28
Elaboracion del modelo de los requerimientos . . . 28
Requerimientos de las negociaciones . . . 28
Validacion de los requerimientos . . . 30
TL;DR . . . 30

Capitulo 6: Modelado de los requerimientos: Escenarios, informacion y clases de analisis 30
Analisis de los requerimientos . . . 30
Objetivos y filosofia general . . . 31
Reglas practicas del analisis . . . 32
Analisis del dominio . . . 32
Enfoques del modelado de requerimientos . . . 32
Modelado basado en escenarios . . . 32
Creacion de un caso preliminar de uso . . . 32
Mejora de un caso de uso preliminar . . . 33
Modelos UML que proporcionan el caso de uso . . . 33
Concepto de modelado de datos . . . 33
Modelado basado en clases . . . 33
Identificacion de las clases de analisis . . . 34
Especificacion de atributos . . . 34
Definicion de las operaciones . . . 34
Modelado clase-responsabilidad-colaborador(CRC) . . . 34
Asociaciones y dependencias . . . 35
Paquete de analisis . . . 35
TL:DR . . . 35

Capitulo 1: Software y la ingenieria de software

Naturaleza del software

Definicion

El software es:

1. Instrucciones (programas de cómputo) que cuando se ejecutan proporcionan las características, función y desempeño buscados.
2. Estructuras de datos que permiten que los programas manipulen en forma adecuada la información.
3. Información descriptiva tanto en papel comoen formas virtuales que describen la operación y uso de los programas.

Caracteristicas que difieren del hardware

- El software se desarrolla o modifica con intelecto; no se manufactura en el sentido clásico.
- El software no se "desgasta".
- Aunque la industria se mueve hacia la construcción basada en componentes, la mayor parte del software se construye para un uso individualizado.

Categorias

Hoy en dia hay siete grandes categorias:

- Software de sistemas
- Software de aplicacion
- Software de ingenierias y ciencias
- Software incrustado
- Software de linea de productos
- Aplicaciones web
- Software de inteligencia artificial

En ciertos casos se elabora software nuevo, pero muchas veces simplemente se corrige, adapta y mejora aplicaciones ya existentes. En base a eso, aparecieron nuevos desafios:

- Computacion en un mundo abierto(computacion distribuida)
- Construccion de redes(poder crear arquitecturas sensillas para algo tan complejo)
- Fuente abierta(el codigo debe ser autodescriptivo para que sea mas sencillo colaborar)

Software heredado

Se denomina **software heredado** a los programas mas antiguos que siguen siendo utilizados, son costosos de mantener y muy dificiles de hacerlos evolucionar.

Este tipo presenta caracteristicas como las mencionadas anteriormente y tambien se le agrega que en general es de mala calidad.

- No facil de actualizar
- Codigo confuso
- Documentacion mala o inexistente

- Casos y pruebas que nunca se archivaron
- Historial de cambios mal administrada

Podria dejarse como esta y seguira funcionando, pero puede ser necesario actualizarlo si:

- El software debe adaptarse para que cumpla las necesidades de los nuevos ambientes del cómputo y de la tecnología.
- El software debe ser mejorado para implementar nuevos requerimientos del negocio.
- El software debe ampliarse para que sea operable con otros sistemas o bases de datos mmodernos.
- La arquitectura del software debe rediseñarse para hacerla viable dentro de un ambiente de redes.

Naturaleza unica de las webapps

La mayoria presentan los siguientes atributos:

- Uso intensivo de redes
- Concurrencia
- Rendimiento
- Disponibilidad
- Orientada a los datos
- Contenido sensible
- Evolucion continue(por ejemplo, youtube, estado del subte)
- Inmediatez(Llegan rapidamente al mercado)
- Seguridad
- Estetica

Ingenieria de software

- El software esta en todos los aspectos de nuestras vidas, muchos clientes. Se concluye que debe hacerse un esfuerzo concertado para entender el problema antes de desarrollar una aplicación de software.
- Las demandas son cada vez mas complejas. Se concluye que el diseño se ha vuelto una actividad crucial.
- Se depende cada vez mas del software para tomar decisiones. Se concluye que el software debe tener alta calidad.
- Al aumentar su valor percibido, aumenta la cantidad de usuarios. Se concluye que el software debe tener facilidad para recibir mantenimiento.

Definicion

La ingeniería de software es una tecnología con varias capas. Cualquier enfoque de ingeniería (incluso la de software) debe basarse en un compromiso organizacional con la calidad. La administración total de la calidad, Six Sigma y otras filosofías similares 10 alimentan la cultura de mejora continua, y es esta cultura la que lleva en última instancia al desarrollo de enfoques cada vez más eficaces de la ingeniería de software. El fundamento en el que se apoya la ingeniería de software es el compromiso con la calidad.

- **Proceso**: El proceso de software forma la base para el control de la administración de proyectos de software, y establece el contexto en el que se aplican métodos técnicos, se generan productos del trabajo (modelos, documentos, datos, reportes, formatos, etc.), se establecen puntos de referencia, se asegura la calidad y se adminis- tra el cambio de manera apropiada.

- **Metodos**: Los métodos de la ingeniería de software se basan en un conjunto de principios fundamentales que gobiernan cada área de la tecnología e incluyen actividades de modelación y otras técnicas descriptivas.
- **Herramientas**: Proporcionan un apoyo automatizado o semiautomatizado para el proceso y los métodos. Cuando se integran las herramientas de modo que la información creada por una pueda ser utilizada por otra, queda establecido un sistema llamado ingeniería de software asistido por computadora que apoya el desarrollo de software.

Proceso del software

Conjunto de actividades, acciones y tareas que se ejecutan al crear algun producto:

- **Actividad**: Busca lograr un objetivo amplio.
- **Accion**: Conjunto de tareas que producen un producto(diseno de arquitectura).
- **Tarea**: Objeto pequeno, pero bien definido(prueba unitaria), produce resultado tangible.

El proceso no es una receta a seguir, sino que se adapta al equipo de trabajo. La estructura del proceso esta formada por las siguientes **actividades estructurales**:

- Comunicacion(con el cliente)
- Planeacion
- Modelado(para entender mejor los requerimientos)
- Construccion
- Despliegue(entrega al consumidor)

Estan son la base, sirven para proyectos sencillos, pero para software mas grande se vuelve mas complejo.

Las actividades estructurales del proceso de ingeniería de software son complementadas por cierto número de **actividades sombrilla**. Estas son:

- Seguimiento y control del proyecto de software
- Administración del riesgo
- Aseguramiento de la calidad del software
- Revisiones técnicas
- Medicion
- Administración de la configuración del software
- Administración de la reutilización
- Preparación y producción del producto del trabajo

  Nota: Un proceso adoptado para un proyecto puede ser significativamente distinto de otro adoptado para otro proyecto.

Practica de la ingenieria de software

Escencia de la practica

Para poder resolver un problema, Polya plantea lo siguiente:

1. Entender el problema (comunicación y análisis).
2. Planear la solución (modelado y diseño del software).
3. Ejecutar el plan (generación del código).

4. Examinar la exactitud del resultado (probar y asegurar la calidad).

Principios generales

Estos ayudan a establecer herramientas mentales para una practica solido. Hooker porpone siete principios:

1. Lo razon de que exista todo, debe dar valor a sus usuarios.
2. KISS(Keep It Simple, Stupid), MSE en castellano.
3. Mantener la vision, una visión clara es esencial para el éxito de un proyecto de software.
4. Otros consumiran lo que usted produce, siempre establezca especificaciones, diseñe e implemente con la seguridad de que alguien más tendrá que entender lo que usted haga.
5. Abrase al futuro
6. Plantee por anticipado la reutilizacion, no reinventar la rueda
7. Piense! Pensar en todo con claridad antes de emprender la acción casi siempre produce mejores resultados.

   Nota: Si todo ingeniero y equipo de software tan sólo siguiera los siete principios de Hooker, se eliminarían muchas de las dificultades que se experimentan al construir sistemas complejos basados en computadora.

Capitulo 2: Modelos del proceso

Se define proceso del software como una estructura para las actividades, acciones y tareas que se requieren a fin de construir software de alta calidad.

Modelo general de proceso

Dentros de las actividades sobrilla se realizan las actividades estructurales, y el conjunto de actividades sobrilla forman la estructura del proceso. Sin embargo, falta mencionar el flujo, escribe la manera en que están organizadas las actividades estructurales y las acciones y tareas que ocurren dentro de cada una con respecto de la secuencia y el tiempo.

Hay varias maneras de realizar el flujo:

- Un **flujo de proceso lineal** realiza las cico actividades estructurales en secuencia.
- Un **flujo de proceso evolutivo** realiza las actividades en forma circular.
- Un **flujo de proceso paralelo** ejecuta una o mas actividades en paralelo.

Definicion: Actividad estructural

¿qué acciones son apropiadas para una actividad estructural, dados la naturaleza del problema por resolver, las características de las personas que hacen el tra bajo y los participantes que patrocinan el proyecto?

Por ejemplo: Para un proyecto sencillo:

1. Hacer contacto con el participante por vía telefónica.
2. Analizar los requerimientos y tomar notas.
3. Organizar las notas por escrito en una formulación breve de los requerimientos.
4. Enviar correo electrónico al participante para que revise y apruebe.

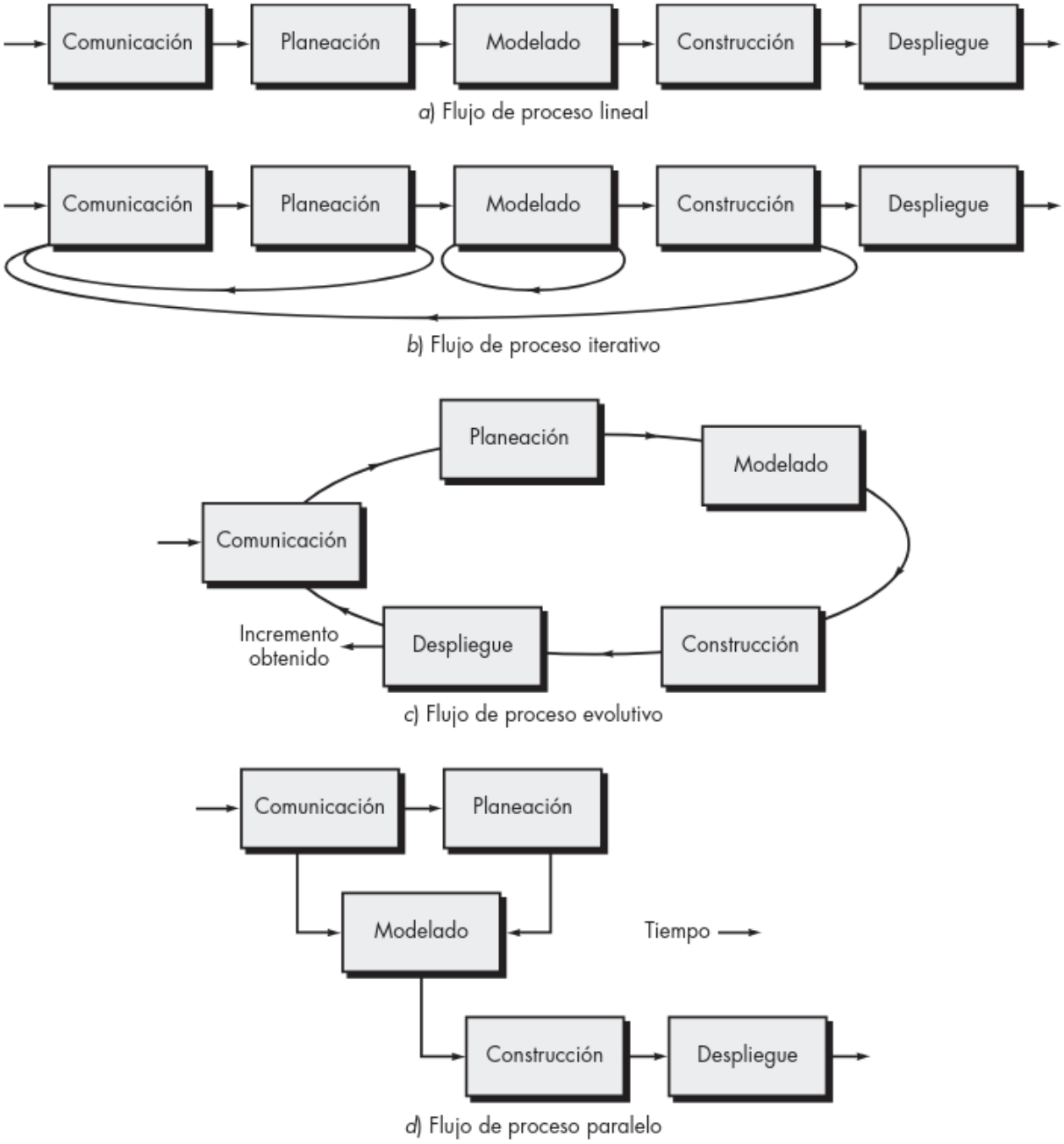

Figure 1: Tipos de flujo

Identificacion de un conjunto de tareas

Debe escogerse el conjunto de tareas que se adapte mejor a las necesidades del proyecto y a las características del equipo. Esto implica que una acción de la ingeniería de software puede adaptarse a las necesidades específicas del proyecto de software y a las características del equipo del proyecto.

Patrones del proceso

Cada equipo se enfrenta a problemas conforme avanza el desarrollo, un **patrón del proceso** describe un problema relacionado con el proceso que se encuentra durante el trabajo de ingeniería de software, identifica el ambiente en el que surge el problema y sugiere una o más soluciones para el mismo.

Amber propuso un formato para describir un patron de proceso:

- Nombre del patron
- Fuerzas
- Tipo(patron de etapa, patron de tarea o patron de fase)
- Contexto inicial
- Problema
- Solucion
- Contexto resultante
- Patrones relacionados
- Usos y ejemplos conocidos

Dan un mecanismo efectivo para enfrentar problemas asociados con cualquier proceso del software. Los patrones permiten desarrollar una descripción jerárquica del proceso, que comienza en un nivel alto de abstracción (un patrón de fase).

Evaluacion y mejora del proyecto

La existencia de un proceso no garantiza que el software se entregue a tiempo o que cumpla con lo pedido. Para evitar esto, el proceso en si puede evaluarse para garantizar que por lo menos cumpla con ciertos criterios que se haya demostrado que son escenciales para el exito.

Algunos enfoques para la evaluacion y mejora son:

- **Metodo de evaluacion del estandar CMMI para el proceso de mejora(SCAMPI)**: Modelo de cinco fases para evaluar el proceso.
- **Evaluacion basada en CMM para la mejora de proceso interno(CBA IPI)**: Tecnica de diagnostico para evaluar madurez relativa.
- **SPICE(ISO/IEC 15504)**: Estandar, define requerimientos para evaluar el proceso. Ayuda a tener una evaluacion objetiva.
- **ISO9001**: Estandar generico que se enfoca en mejorar la calidad de los productos, directamenta aplicable al desarrollo de software.

  Nota: CMMI significa Capability Maturity Model Integration. SEI CMMI y SEI CMM son tecnicas de evaluacion.

Modelos del proceso prescriptivo

Propuestos para poner orden, an resultado ser muy utiles, pero aplicado al software el producto que genera sigue estando "al borde del caos". A continuacion se mustran algunos de estos modelos:

- Modelo de la cascada
- Modelo de proceso incremental
- Modelo de proceso evolutivo(Involucra el modelo espiral)
- Modelos concurrentes

El reto principal es establecer un balance apropiado entre estos parametros criticos del proyecto y el producto, y la satisfaccion del cliente(el que tendra la palabra final con respecto a la calidad).

> **Nota**: Descripcion de estos modelos estan en los apuntes de clase.

Modelo de procesos especializado

Comparten algunas caracteristicas con los modelos tradicionales, pero tienden a aplicarse cuando se tiene un enfoque muy especifico.

Desarrollo basado en componentes

Incorpora muchas de las caracteristicas del modelo espiral, es de naturaleza evolutiva y demanda enfoque iterativo. Su principal caracteristica es que **contruye aplicaciones a partir de fragmentos de software prefabricados**.

Incorpora las siguientes etapas:

1. Se investiga y evaluan productos ya disponibles.
2. Se consideran aspectos de integracion.
3. Se disena una arquitectura que reciba los coomponentes.
4. Se integran los componentes a la arquitectura.
5. Se efectuan pruebas para asegurar la funcionalidad adecuada.

Este modelo lleva a la reutilizacion del software, reduciendo el tiempo y costo del desarrollo.

Modelo de metodos formales

Tiene un enfoque matematico, desarrollando un sistema por medio de una notacion matematica. Lo ambiguo o incorrecto se corrije con mucha mas facilidad mediante el analisis matematico. Aunque no es el modelo mas seguido, promueve software libre de defectos. Ademas, permite construir software de muy alta calidad en terminos de seguridad o donde los errores son muy costosos.

Sin embargo, tiene sus desvantajas:

- El desarrollo consume mucho tiempo y es caro.
- Se requiere mucha capacitacion del personal.
- Es dificil de implementar con clientes que no tienen la complejidad tecnica.

> **Nota**: Existe una variante de este enfoque que se denomina *ingenieria de software de quirofano*.

Desarrollo orientado a aspectos

Sin importar el proceso elejido, estos tienen varias caracteristicas en comun. A medida que se hacen mas sotisficados los proyectos, el desarrollo de software presenta **preocupaciones globales**. Afectan multiples funciones, caracteristicas e informacion del sistema.

Por lo tanto, el DSOA es un paradigma nuevo que se enfoque en definir, especificar y construir aspectos, estos pueden ser de:

- Persistencia(almacenamiento)
- Seguridad
- Transacciones(control de concurrencia, por ejemplo)

El proceso unificado

La tendencia en el software es hacia sistemas mas grandes y complejos, principalmente a que las computadoras son cada ves mas potentes. El proceso unificado es un intento de obtener mejores caracteristicas de los modelos tradicionales, pero que a su vez implemente los mejores principios del desarrollo **agil**. Sugiere flujo iterativo e incremental, lo que da la sensacion evolutiva que resulta esencial en el desarrollo.

Fases del proceso unificado

Estas faces involucran las **actividades del proceso** mencionadas anteriormente, mostrando como pueden usarse para describir cualquier modelo.

- **Fase de concepcion**: Comunicacion con cliente y planeacion.
- **Fase de elaboracion**: Comunicacion y modelado del modelo general de proceso. Es comun que al finalizar se hagan modificaciones al plan.
- **Fase de construccion**: Actividad de construccion. Desarrolla y adquiere componentes del software(por ejemplo, el lanzamiento). Pruebas unitarias, integracion, etc.
- **Fase de transicion**: Ultimas etavas de la actividad de construccion y primera parte de despliegue general(entrega y feedback). Se da a los usuarios el software de prueba. Se genera informacion de apoyo(manuales, por ejemplo).
- **Fase de produccion**: Coincide con la actividad de despliegue del proceso general. Se reportan defectos y solicitudes.

El **flujo de trabajo** de la ingenieria de softare esta distribuido en todas las fases del PU. Sin embargo, no toda tarea mencionada es necesariamente realizada en todos los proyectos de software(se adapta al equipo).

> **Nota**: Las fases no ocurren en secuencia, sino de forma escalonada.

Modelos del proceso personal y del equipo

El proceso esta cerca de las personas que haran el trabajo, si el modelo se realiza a nival corporativo, sera efiscaz solo si acepta una adaptacion significativa. Idealmente, el proceso debe ajustarse del mejor modo a los requerimientos y cumbrir, al mismo tiempo, las amplias necesidades del equipo y de la organizacion. Alternativamente, el equipo puede ser el que se encargue de realizar el proceso, enfocandoce mas en el equipo y solo cubrir las necesidades generales de la organizacion.

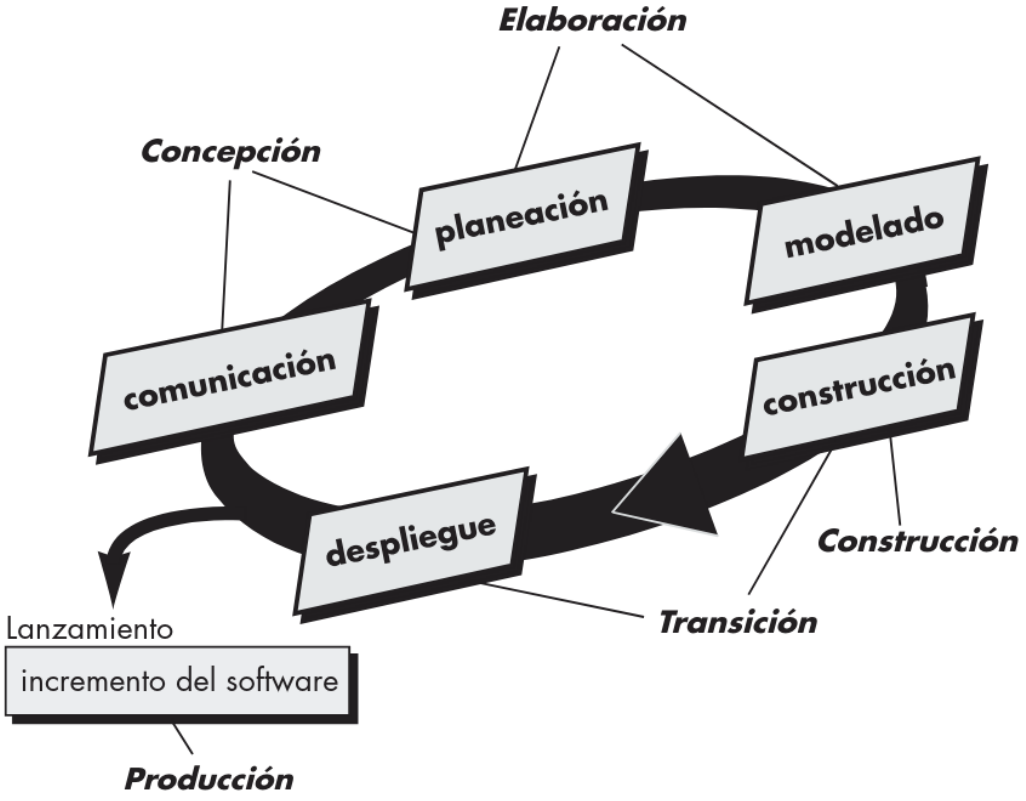

Figure 2: Fases del proceso unificado

Proceso personal del software(PPS)

Se enfoca en la medicion personal tanto del producto del trabajo que se genera como de su calidad, responsabiliza al profesional acerca de la planeacion del proyecto(por ejemplo, programacion de actividades) y delega en el practicante la calidad de los productos. Define cinco actividades estructurales:

- **Planeacion**: Aisla requerimientos y desarrolla estimaciones(numero de defectos, recursos, etc.). Se identifican las tareas de desarrollo y se crea un programa.
- **Diseno de alto nivel**: Especificaciones externas para cada componente, si hay incertidumbre se elaboran prototipos.
- **Revision de diseno de alto nivel**: Se aplican metodos de verificacion formal para descubrir errores. Se miden las tareas y resultados.
- **Desarrollo**: Mejora y revisa el diseno, se genera el codigo, se revisa y prueba. Se miden las tareas y trabajos de importancia.
- **Post mortem**: Se determina la eficacia del proceso por medio de mediciones(nuevas o recolectadas). Se mejora la eficacia.

Se centra en detectar errores lo antes posible, no a sido adoptado con amplitud en la industria debido a que es culturamente dificil realizar el nivel requerido de mediciones.

Proceso del equipo de software(PES)

El objetivo de este es construir un equipo **autodirigido** para el proyecto, que se organice para producir software de alta calidad. Tiene los siguientes objetivos:

- Formar equipos autodirigidos.
- Mostrar a los gerentes como dirigir y motivar a sus equipos.
- Acelerar la mejora del proceso de software(CMM).
- Brindar a las organizacion una guia para la mejora.
- Facilitar la ensenanza universitaria de amplitudes de equipo.

Un equipo autodirigido tiene una comprension consistente de sus metas y objetivos, debe haber colaboracion interna y comunicacion externa. Define las siguientes actividades estructurales(similar a PPS):

- Inicio del proyecto
- Diseno de alto nivel
- Implementacion
- Integracion y pruebas
- Post mortem

Utiliza amplia variedad de **scripts**, los cuales definen formatos y estandares que guian a los miembros del equipo. Reconoce que los mejores equipos de software son los autodirigidos, los miembros son los responsables de seguir con los objetivos y adaptarse en caso de ser necesario.

Tecnologia del proceso

El equipo debe adaptar uno o mas de los modelos de proceso mencionados, para ello existen **herramientas de tecnologia del proceso**. Una vez creado un proceso aceptable, se emplean otras herramientas de tecnologia para asignar, vigilas e incluso controlas todas las actividades. Tambien puede ser usada para

coordinar el empleo de otras herramientas que sean apropiadas para una tarea en particular. Margaret Davis hace comentarios sobre la dualidad del producto y del proceso:

> En tanto que la tendencia natural de un péndulo es alcanzar el estado de reposo en el punto medio entre dos extremos, la atención de la comunidad del software cambia constantemente porque se aplica una nueva fuerza al fallar la última oscilación. Estos vaivenes son dañinos en sí mismos porque confunden al profesional promedio del software al cambiar en forma radical lo que significa hacer el trabajo bien. Los cambios periódicos no resuelven "el problema" porque están predestinados a fallar toda vez que el producto y el proceso son tratados como si fueran una dicotomía en lugar de una dualidad.
>
> Obtenemos sentimientos de satisfacción por la reutilización de nuestros productos, ya sea que lo hagamos nosotros u otras personas.

Como profesional creativo, hay que ser capaz de obtener tanta satisfaccion del proceso como del producto final. La dualidad planteada es un elemento importante para hacer que personas creativas se involucren.

TL;DR

Un modelo general del proceso incluye un conjunto de actividades, cada uno de los modelos puede describirse por un flujo distinto. Los patrones del proceso pueden utilizarse para resolver problemas comunes que surgen durante el proceso. Se dividen en 3 grandes grupos:

- Modelos de proceso prescriptivo
- Modelos de proceso secuencial
- Modelos de proceso incremental

Capitulo 24: Conceptos de administracion de proyecto

Si un proyecto no se cumple a tiempo o contraria a sus usuarios, esto se debe generalmente a una mala administracion. En este capitulo se desarrolla como se puede llegar a una administracion efectiva.

El espectro administrativo

El trabajo de la ingenieria de software es una empresa intensamente humana, fracasar en una comunicacion comprensiva durante las primeras etapas de un proyecto arriesga a construir una solucion para el problema equivocado.

La administracion efectiva se enfoca en(De mayor a menor prioridad):

1. **Personal**: El factor humano es tan importante que existe **People-CMM**(Modelo de madurez de capacidades del personal). Define areas practicas clave: plantilla, comunicación y coordinación, ambiente de trabajo, desempeño administrativo, capacitación, compensación, análisis y desarrollo de competencias, desarrollo profesional, desarrollo de grupo de trabajo y desarrollo de equipo/cultura, entre otros.
2. **Producto**: Antes de planear se debe establecer objetivos(metas globales) y el ambito del producto(datos, funciones, comportamientos), considerarse soluciones alternativas e identifiar restricciones. Todos los participantes deben reunirse para definir estas cosas, esta actividad comienza como parte de la ingenieria del sistema y continua con la ingenieria de requerimientos.

3. **Proceso**: Proporciona el marco conceptual, un pequeno numero de actividades se esta area se aplica a todos los proyectos de software, sin importar el tamano o complejidad. Algunos conjuntos de tareas(hitos, tareas, produtos operativos) permiten que las actividades del marco conceptual se adapten al proyecto. Mientras que las actividades sombrilla recubren el modelo de proceso(independientes de cualquier actividad del marco).
4. **Proyecto**: Se planean y controlan debido a que es la unica forma de manejar su complejidad(aun asi, son dificiles). Para evitar el fracaso, se debe evitar un conjunto de advertencias, entender los factores de exito cruciales y desarrollar un enfoque de sentido comun.

El personal

El factor mas importante, segun administradores de grandes companas, es el personal, pero con frecuencia el personal se da por hecho dentro de las empresas.

Los participantes

Los participantes de un proyecto pueden organizarse dentro de las siguientes areas:

- **Gerentes ejecutivos**: Definen temas empresariales.
- **Gerentes de proyecto(tecnicos)**: Planifican, motivan, organizan y controlan a los profesionales.
- **Profesionales8**: Aportan habilidades tecnicas.
- **Clientes**: Especifican requerimientos.
- **Usuarios finales**: Interactuan una vez se libera el software.

Es responsabilidad del lider que las personas esten organizadas de manera que se maximicen sus habilidades.

Lideres de equipo

Administrar implica trato con gente, debido a eso los profesionales competentes tienen pobre desempeno como lideres de equipo, pero generalmente uno no elije el papel de lider, sino que se topa con el.

Se sugiere el modelo **MOI** de liderazgo:

- Motivacion(alentar al personal)
- Organizacion
- Ideas o innovacion(alentar a crear)

Por otro lado, un gerente eficaz debe tener los siguientes rasgos:

- Resolucion de problemas
- Indentidad administrativa
- Logro
- Influencia y construccion del equipo

Equipo de software

La mejor estructura del equipo depende del estilo administrativo, del numero de personas y de sus niveles de habilidad, asi como la complejidad del problema. Hay siete factores que deben considerarse cuando se plantee la estructura:

- Dificultad del problema que se va a resolver.

- “Tamaño” del programa resultante en líneas de código o puntos de función.
- Tiempo que el equipo permanecerá unido (vida del equipo).
- Grado en el que puede dividirse en módulos el problema.
- Calidad y confiabilidad requeridas por el sistema que se va a construir.
- Rigidez de la fecha de entrega.
- Grado de sociabilidad (comunicación) requerido para el proyecto.

Paradigmas organizacionales sugeridos:

- **Paradigma cerrado**: Jerarquia de autoriadad, trabajan bien cuando producen software similar a esfuerzos anteriores, menos innovadores.
- **Paradigma aleatorio**: Estructura amplia, depende de la iniciativa individual. Se destaca cuando se necesita innovar.
- **Paradigma abierto**: Intenta de tener algunos de los controles del paradigma cerrado, pero tambien mucha innovacion. Se trabaja de manera colaboradora y es adecuado para problemas complejos, no tiene tanto desempeno como otros equipoc.
- **Paradigma sincrono**: Compartimentalizacion natural de un problema, los miembros trabajan en trozos con poca comunicacion entre ellos.

Para lograr un equipo de alto rendimiento:

- Los miembros del equipo deben tenerse confianza entre sí.
- La distribución de habilidades debe ser adecuada para el problema.
- Es posible que tenga que excluirse del equipo a los inconformes si debe mantenerse la cohesión del equipo.

Sin importar la organizacion, todo gerente de proyecto debe ayudar a crear un equipo que muestre cohesion. Un equipo** cuajando** es un equipo donde los miembros estan muy fuertemente unido, trabajan muy bien juntos y el producto es mayor que la suma de sus partes. Los miembros de un equipo cuajando son mas productivos y motivados que el promedio.

Lamentablemente, no todos los equipos cuajan, muchos sufren toxicidad. Factores que fomentan toxicidad:

- Atmosfera de trabajo frenetico
- Alta frustracion
- Proceso fragmentado o pobremente fraccionado
- Definicion poco clara de roles
- Continua y repetida exposicion al fracaso

Un equipo de software puede evitar la frustración si se le da tanta responsabilidad para la toma de decisiones como sea posible. nalmente, la clave para evitar una atmósfera de fracaso radica en establecer técnicas basadas en equipo para retroalimentarse y resolver problemas.

Ademas, un equipo de software con frecuencia batalla con los diferentes rasgos humanos de sus miembros. Algunos son extrovertidos; otros, introvertidos. Algunas personas reúnen información de manera intuitiva y separan los conceptos abarcadores de los hechos dispares. Otras procesan la información de manera lineal, y reúnen y organizan detalles minúsculos de los datos proporcionados. Es importante observar que el reconocimiento de las diferencias humanas es el primer paso hacia la creación de equipos que cuajen.

Equipos agiles

Es un equipo enormemente motivado y preparado para trabajar exitosamente. Son equipos **autoarganizados**, no necesariamente mantiene una sola estructura, sino que usa elementos de los paradigmas aleatorio, abierto y sincronico.

Con base en la información obtenida durante dichas reuniones, el equipo adapta su enfoque para lograr un incremento de trabajo. Conforme transcurre cada día, la autoorganización y la colaboración continuas mueven al equipo hacia un incremento de software completo.

Conflictos de coordinacio y comunicacion

El software moderno presenta tres caracteristicas principales: escala, incertidumbre e interoperabilidad. Para lidiar con esto, deben implementarse metodos efectivos. Mediante mecanismos que ayuden con la comunicacion formal e informal. Los miembros de un equipo de software comparten ideas sobre una base *ad hoc*, piden ayuda cuando surgen problemas e interactúan unos con otros diariamente.

El producto

Se presenta un dilema al comienzo: se necesitan estimaciones y un plan, pero no hay informacion disponible. Un analisis de los requerimientos tarda semanas o meses en completarse, y estos pueden cambiar con regularidad. Por lo tanto, el gerente debe examinar el producto, para luego establecer y acotar el ambito del producto.

Ambito del software

La primera actividad en la administracion de un proyecto, responde las siguientes preguntas:

- Contexto
- Objetivos de informacion(Datos de entrada y de salida)
- Funcion y desempeno

No debe tener ambiguedades ni ser incomprensible. Debe acotar.

Descomposicion del problema

Division o elaboracion del problema, se encuentra dentro del analisis de requerimientos del software. No se hacen intentos por descomponer el problema, sino que se descompone:

- La funcionalidad y el contenido que deben entregarse
- El proceso que se usara par aentregarlo

El proceso

El problema es seleccionar el modelo de proceso(capitulo 2) que sea adecuado para el software. Se debe elejir el modelo que sea adecuado para:

- Clientes que solicitaron el producto y el personal que hara el trabajo
- Caracteristicas del producto
- Entorno del proyecto donde trabaja el equipo

Una vez seleccionado, el equipo define un plan de proyecto preeliminar. Luego comienza la descomposicion, creando un plan completo que refleje las tareas requeridas para completar las actividades del marco conceptual.

Fusion de producto y proceso

La planificacion comienza con esta fusion. Cada función que se va a someter a ingeniería por parte del equipo debe pasar a través del conjunto de actividades de marco conceptual que defina la organización de software.

Descomposicion del proceso

Un equipo de software debe tener un grado significativo de flexibilidad al elegir el modelo de proceso de software que es mejor para el proyecto y las tareas que pueblen el modelo.

Una vez elegido el modelo, el marco conceptual se adapta a el. Funcionara para los modelos lineales, iterativos, incrementales, evolutivos e incluso para modelos concurrentes o de ensamble. El marco conceptual es invariante y sirve como base.

Sin embargo, las tareas si varian, la descomposicion comienza cuando el gerente cuestiona como van a lograr alguna actividad del marco conceptual.

El proyecto

Para administrar un proyecto de software exitoso, se debe comprender qué puede salir mal, de modo que los problemas puedan evitarse. Hay 10 indicadores de que un proyecto esta en peligro:

1. El personal del software no entiende las necesidades del cliente.
2. El ámbito del producto está pobremente definido.
3. Los cambios se gestionan pobremente.
4. Cambia la tecnología elegida.
5. Las necesidades empresariales cambian [o están mal definidas].
6. Las fechas límite son irreales.
7. Los usuarios son resistentes.
8. Pérdida de patrocinio [o nunca obtenido adecuadamente].
9. El equipo del proyecto carece de personal con habilidades adecuadas.
10. Los gerentes [y profesionales] evitan mejores prácticas y lecciones aprendidas.

    Nota: Se suele referirse a la regla 90-90, donde el ultimo 10% es el 90% del esfuerzo.

Para evitar estos problemas, se sugiere el siguiente enfoque:

- Comenzar con el pie derecho, trabajar muy duro para entender el problema.
- Mantener la contidad de movimiento, minima rotacion de personal y el administrador ejecutivo debe permanecer fuera del camino del equipo.
- Siga la pista al progreso, para asegurar calidad se rastrea el progreso.
- Tome decisiones inteligentes, deben mantenerse simples.
- Realice un analisis postmortem, extrear lecciones aprendidas.

Principio W^5HH

Se trata de una serie de preguntas que conducen a una definicion de caracteristticas clave y al plan de proyecto resultante:

- Why
- What
- When
- Who
- Where
- How
- How much

Es plicable sin importar el tipo de proyecto.

Practicas cruciales

Practicas utilizadas por proyectos y organizaciones enormemente exitosas:

- Administracion del proyecto basada en metrica
- Rastreo de defecto contra metas de calidad
- Administracion consciente del personal

TL;DR

La administración de proyectos de software es una actividad sombrilla dentro de la ingeniería de software. Comienza antes de iniciar cualquier actividad técnica y continúa a lo largo del modelado, construcción y despliegue del software de cómputo.

En general, las revisiones técnicas y la comunicación informal persona a persona tienen más valor para los profesionales. El elemento esencial en todos los proyectos de software es el personal.

Capitulo 3: Desarrollo agil

Algunos autores creen que los procesos prescriptivos tienen una gran desventaja y es que:

> Olvidan las flaquezas de las personas cuando construyen software. Los ingenieros de software no son robots. Sus estilos de trabajo varían mucho; tienen diferencias significativas en habilidad, creatividad, orden, consistencia y espontaneida.
>
> Como la consistencia de las acciones es una debilidad humana, las metodología que requieren mucha disciplina son frágiles.

Manifesto agil

- Los **individuos y sus interacciones**, sobre los procesos y las herramientas
- El **software que funciona**, más que la documentación exhaustiva
- La **colaboración con el cliente**, y no tanto la negociación del contrato
- **Responder al cambio**, mejor que apegarse a un plan

Que es la agilidad?

Aun mas especificamente, que es la agilidad en el contexto del desarrollo de software. Es algo mas que una respuesta efectiva al cambio, incluye la filosofia descripta en el manifesto. Pone el énfasis en:

- La entrega rápida de software funcional y resta importancia a los productos intermedios del trabajo (lo que no siempre es bueno).
- Adopta al cliente como parte del equipo de desarrollo y trabaja para eliminar la actitud de "nosotros y ellos" que todavía invade muchos proyectos de software.
- Reconoce que la planeación en un mundo incierto tiene sus límites y que un plan de proyecto debe ser flexible.

Es importante para poder aplicar la agilidad que se disene el proceso de manera que:

- Permita al equipo adaptar las tareas.
- Ejecutar la planeacion de manera fluida.
- Dejar solo los productos del trabajo esenciales.
- Poner enfasis en una estrategia de entrega incremental.

La agilidad y el costo del cambio

Empiricamente te puede observar que el costo de un proyecto evoluciona de forma no lineal a medida que el proyecto avanza. En especial si se requiere ahcer un cambio en una etapa avanzada, donde es muy costoso y lleva mucho tiempo evitar que se haga un cambio sin efectos colaterales no intencionados.

Un proceso agil bien disenado aplana el costo de la curva de cambio(ver imagen).

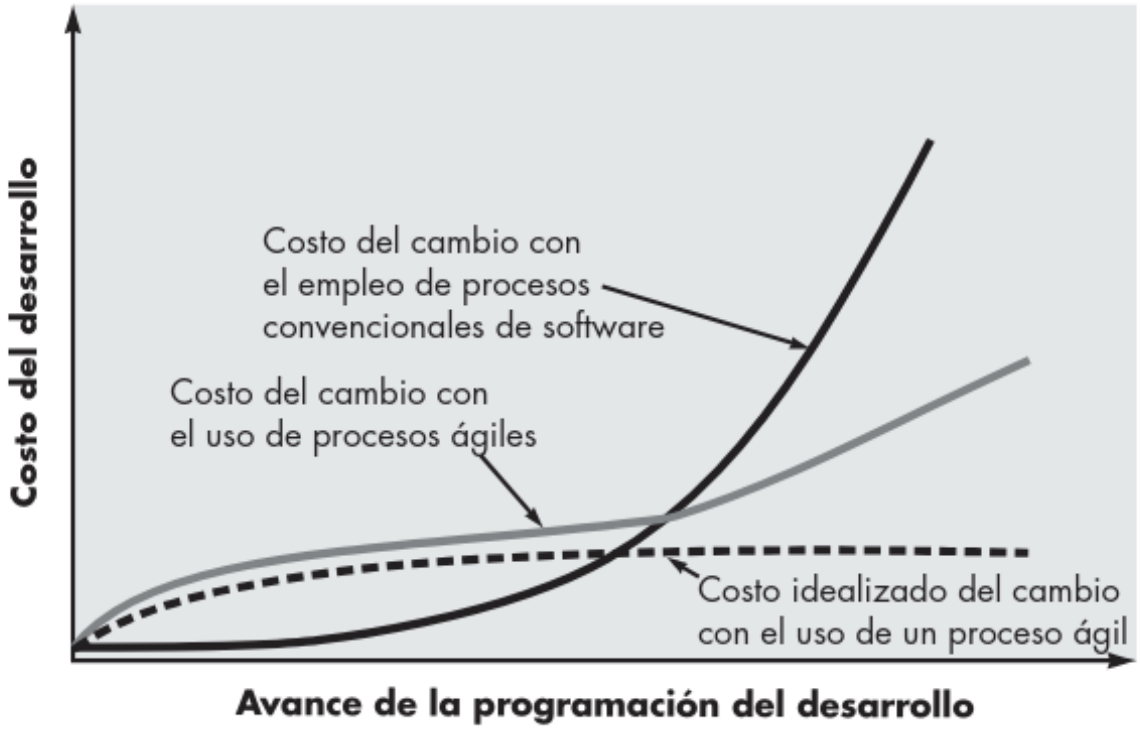

Figure 3: Curva de costo durante un programa de desarrollo

Cuando ésta se acopla con otras prácticas ágiles, como las pruebas unitarias continuas y la programación por parejas (que se estudia más adelante, en este capítulo), el costo de hacer un cambio disminuye.

Que es un proceso agil?

Cualquier proceso del software ágil se caracteriza por la forma en la que aborda cierto número de suposiciones clave:

- Es dificil predecir como cambiaran los requisitos.
- El diseno y la construccion deben ejecutarse, generalmente, de forma simultanea.
- El análisis, el diseño, la construcción y las pruebas no son tan predecibles como nos gustaría.

Un proceso agil debe:

- Adaptarse incrementalmente(con feedback del cliente).
- Instuirse una estrategia de desarrollo incremental.
- Entregar incrementos de software(prototipos o porciones).

Principios de agilidad

Se definen 12 principios de agilidad para aquellos que la quieran alcanzar:

1. La prioridad más alta es satisfacer al cliente a través de la entrega pronta y continua de software valioso.
2. Son bienvenidos los requerimientos cambiantes, aun en una etapa avanzada del desarrollo.
3. Entregar con frecuencia software que funcione, de dos semanas a un par de meses, de preferencia lo más pronto que se pueda.
4. Las personas de negocios y los desarrolladores deben trabajar juntos, a diario y durante todo el proyecto.
5. Hay que desarrollar los proyectos con individuos motivados. Debe darse a éstos el ambiente y el apoyo que necesiten, y confiar en que harán el trabajo.
6. El método más eficiente y eficaz para transmitir información a los integrantes de un equipo de desarrollo, y entre éstos, es la conversación cara a cara.
7. La medida principal de avance es el software que funciona.
8. Los procesos ágiles promueven el desarrollo sostenible. Los patrocinadores, desarrolladores y usuarios deben poder mantener un ritmo constante en forma indefinida.
9. La atención continua a la excelencia técnica y el buen diseño mejora la agilidad.
10. Es esencial la simplicidad: el arte de maximizar la cantidad de trabajo no realizado.
11. Las mejores arquitecturas, requerimientos y diseños surgen de los equipos con organización propia.
12. El equipo reflexiona a intervalos regulares sobre cómo ser más eficaz, para después afinar y ajustar su comportamiento en consecuencia.

No todos los modelos agil aplican los 12 principios, pero definen la idea principal y la filosofia de agil.

La politica de desarrollo agil

Hay mucho debate hacerca de la efectividad de agil en comparacion con los metodos tradicionales. Esto se torno en un tema de debate sencible que rapidamente puede llevar a una discusion. Sin embargo, la verdadera pregunta es cual es la mejor forma de implementar agil?

No hay respuestas absolutas a esto, entonces, hay mucho por ganar si se considera lo mejor de ambas escuelas, y virtualmente no se gana nada si se denigra cualquiera de los enfoques.

Factores humanos

Los defensores de agil se enfocan principalmente en los **factores personales**. Si los miembros del equipo de software son los que van a generar las características del proceso que van a aplicarse a la elaboración de software, entre ellos debe existir cierto número de características clave, mismas que debe compartir el equipo ágil como tal:

- Competencia
- Enfoque comun
- Colaboracion
- Habilidad para tomar decisiones
- Capacidad para resolver problemas difusos
- Confianza y respeto mutuos
- Organizacion propia

Programacion extrema(XP)

Es el enfoque mas utilizado en el desarrollo agil, a continuacion se dara un panorama.

Valores

Se define un conjunto de 5 valores:

- Comunicacion
- Simplicidad
- Retroalimentacion
- Valentia
- Respeto

Proceso

Usa un enfoque orientado a objetos como paradigma preferido. Las actividades estructurakes clave son:

- Planeacion
- Diseno
- Codificacion
- Prueba

XP industrial

IXP(Programacion Externa Industrial), es la evolucion organica de XP, tiene un espiritu minimalista, centrado en el cliente y orientado a las pruebas que tiene XP.

IXP incorpora 6 practicas nuevas para ayudar a garantizar que un proyecto XP tenga exito:

- Evaluacion de la factibilidad
- Comunidad del proyecto
- Calificacion del proyecto

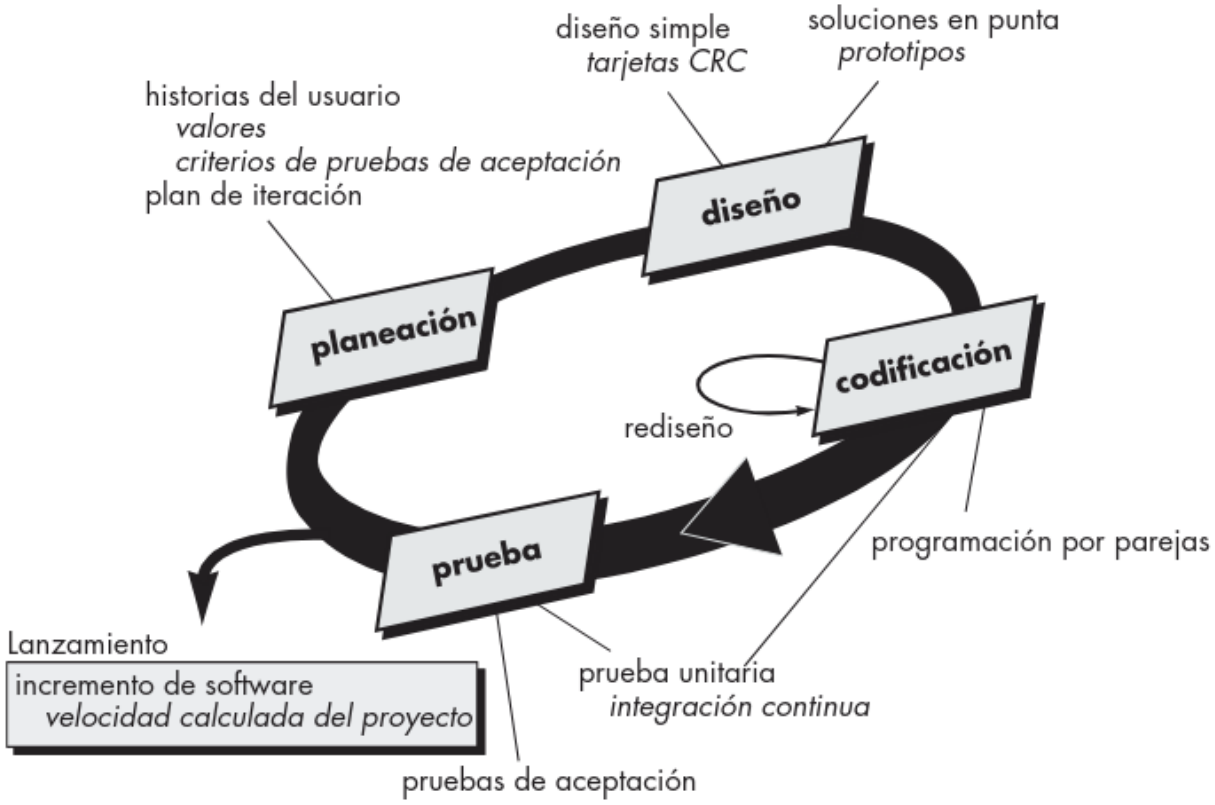

Figure 4: Actividades de XP

- Administracion orientada a pruebas
- Retrospectivas
- Aprendizaje continuo

Ademas, modifica algunas de las existentes en XP. La IXP hace modificaciones más pequeñas a otras prácticas XP y redefine ciertos roles y responsabilidades para hacerlos más asequibles a proyectos significativos de las organizaciones grandes.

Debate XP

Los nuevos modelos y métodos de proceso han motivado análisis provechosos y en ciertas instancias debates acalorados. La programación extrema desencadena ambos. Entre los aspectos que destacan algunos críticos de la XP están los siguientes:

- Volatibilidad de los requerimientos
- Necesidades conflictivas con el cliente(con varios clientes)
- Los requerimientos se expresan informalmente
- Falta de un diseno formal

Otros modelos agiles de proceso

La historia de la ingeniería de software está salpicada de decenas de descripciones y metodologías de proceso, métodos de modelado y notaciones, herramientas y tecnología, todos ellos obsoletos. El mas usado es XP, pero hay mas. Entre ellos se encuentran los siguientes:

- Desarrollo adaptativo de software (DAS)
- Scrum
- Método de desarrollo de sistemas dinámicos (MDSD)
- Cristal
- Desarrollo impulsado por las características (DIC)
- Desarrollo esbelto de software (DES)
- Modelado ágil (MA)
- Proceso unificado ágil (PUA)

Ver libro para una breve descripcion de cada uno.

TL:DR

En una economía moderna, las condiciones del mercado cambian con rapidez, los clientes y usuarios finales necesitan evolucionar y surgen nuevas amenazas competitivas sin aviso previo. Los profesionales deben enfocar la ingeniería de software en forma que les permita mantenerse ágiles para definir procesos maniobrables, adaptativos y esbeltos que satisfagan las necesidades de los negocios modernos.

El pricipal modelo agil es XP, pero otros modelos de proceso ágil también insisten en la colaboración humana y en la organización propia del equipo, pero definen sus actividades estructurales y seleccionan diferentes puntos de importancia.

Capitulo 5: Comprension de los requerimientos

Una de las tareas mas dificiles que enfrenta un ingeniero de software; tiende un punte para el diseno y la construccion.

Tareas

Incluye siete tareas dificiles:

- Concepcion
- Indagacion
- Elaboracion
- Negociacion
- Especificacion
- Validacion
- Administracion

Algunas de estas tareas ocurren en paralelo y todas se adaptan a las necesidades del proyecto.

Concepcion

Una conversacio casual es todo lo que se necesita para iniciar un proyecto de software, pero en general, comienzan cuando se identifica una necesidad de negocio o se descubre un nuevo mercado o servicio potencial.

En esta parte se establece:

- Entedimiento basico del problema
- Personas que quieren la solucion
- Naturaleza de la solucion que se desea
- Eficacia de comunicacion
- Colaboracion preliminares entres otros participantes y el equipo del software

Indagacion

Parece simple, pero no lo es. No es simplemente preguntar los objetivos para el sistema, sino que surge un cierto numero de problemas:

- Problemas de alcance
- Problemas de entendimiento
- Problemas de volatilidad(requerimientos cambian con el tiempo)

Para superarlos, se debe enfocarse en la obtencion de requerimientos en forma organizada.

Elaboracion

A partir de la informacion obtenida en las dos estapas anteriores, se desarrolla un modelo refinado de los requerimientos que identifique factores clave del software pedido.

Esta motivada por la creacion y mejora de escenarios de usuario que describan como interactua el usuario final con el sistema.

Negociacion

No es raro que el cliente pida mas de lo que puede lograrse, o que propongan requerimientos conflictivos. Estos conflictos se resulven mediante una negociacion, pidiendo que se ordenen los requerimientos por prioridad y luego se analice los conflictos. Luego de elimniar, modificar o combinar requerimientos, cada parte logra un grado de satisfaccion.

Especificacion

Tiene distinto significado para distitas personas en el ambito del software, pero una especificacion suele ser:

- Documento escrito
- Conjunto de modelos graficos
- Modelo matematico formal
- Conjunto de escenarios de uso
- Prototipo
- Cualquier combinacion de los anteriores.

Algunos sugieren utilizar una plantilla estandar para mantener consistencia, pero es necesario poder ser flexible. Para sistemas simples, puede ser suficiente tener solo escenarios de uso.

Validacion

Busca garantizar que los requerimientos no tengan ambiguedades, inconsistencias, omisiones o errores. Sobre todo que se presenten comforme a los estandares establecidos. El mecanismo principal para lograr esto es la **revision tecnica**.

Administracion de los requerimientos

Como los requerimientos suelen cambiar, esta tarea se enfoca en ayudar al equipo del proyecto a identificar, controlar y dar seguimiento a los requerimiento y a sus cambios en cualquier momento.

Establecer las bases

Idealmente los participantes y los ingenieros trabajan juntos, pero no siempre ocurre, quiza solo haya una idea vaga de lo que se requiere y los clientes no esten cerca para consultarlos.

Para que se entiendan bien los requerimientos y el proyecto comience bien encamidado, hay varias etapas requeridas para establecer estas bases:

1. **Identificacion de los participantes**: Cualquier persona que se beneficie en forma directa o indirecta del sistema en desarrollo.
2. **Reconocer los multiples puntos de vista**: Cada participante tiene una vision diferente del sistema.
3. **Trabajar hacia la colaboracion**: Identificar tareas de interes comun y las de conflicto y resolver conflictos usando puntos de prioridad (como una especie de votacion).
4. **Hacer las primeras preguntas**: Las preguntar realizadas en la concepcion deben estar **libres de contexto**, se usan para identificar a los participantes y *romper el hielo* para iniciar la comunicacion.

Indagacion de los requerimientos

Tambien conocida como **recabacion de los requerimientos** combina elementos de la solucion de problemas, elaboracion, negociacion y especificacion.

Recabacion en forma colaborativa

Para ello, se proponen los siguientes lineamientos basicos:

- Tanto ingenieros como participantes dirigen o intervienen en las reuniones
- Se establecen reglas para la preparacion y participacion
- Se sugiere una agenda con suficiente formalidad para cubrir todos los puntos importantes, pero con suficiente informalidad para promover el libre flujo de ideas.
- Un cliente/desarrollador/participante externo (facilitador) controla la reunion
- Se utiliza un *mecanismo de definicion*

La meta es identificar el problema, proponer elementos de la solucion, negociar distintos enfoques y especificar un conjunto preliminar de requerimientos de la solucion.

Despliegue de la funcion de calidad

Tambien conocida como DFC es una tecnica de administracion de la calidad que traduce las necesidades del cliente en requerimiento tecnicos para el software. Se enfoca en entender que es valioso para el cliente, tiene tres tipos de requerimientos:

- **Requerimientos normales**: Si estan presentes, el cliente queda satisfecho.
- **Requerimientos esperados**: Implicitos, pero importantes para el cliente. Por ejemplo, operacion general correcta y confiable.
- **Requerimientos emocionantes**: Mas alla de las espectativas, muy satisfactorias si estan presentes.

Basicamente, en base a los datos obtenidos, se crea una tabla de requerimientos, conocida como **tabla de la voz del cliente** para luego extraer los requerimientos esperados y percibir los emocionantes.

Escenarios de uso

Entender como emplearan los uusuarios finales dichas funciones y caracteristicas. Estos escenarios de llaman **casos de uso**.

Indagacion(Analisis) de los productos de trabajo

Los productos de trabajo son generados a partir de la indagacion de los requerimienots, estos varian en funcion al tamano de proyecto. Generalmente, los productos de trabajo incluyen los siguientes:

- Enunciado de la necesidad y su factibilidad.
- Enunciado acotado del alcance del sistema o producto.
- Lista de clientes, usuarios y otros participantes que intervienen en la indagacion de los requerimientos.
- Una descripcion del ambiente tecnico del sistema.
- Una lista de requerimientos(de preferencia organizados por funcion) y las restricciones del dominio que aplican a cada uno.
- Un conjunto de escenarios de uso que dan perspectiva al uso del sitema o producto en diferentes condiciones de operacion.

- Cualesquiera prototipos desarrollados para definir requerimientos.

Cada uno de estos es revisado por todas las personas que participan en la indagacion de los requerimientos. Esto es una actividad evolutiva.

Desarrollo casos de uso

Un caso de uso narra una historia estilizada sobre como interactua un usuario final con el sistema en un contexto especifico. Ilustra el software desde el punto de vista del usuario final. Tiene los siguientes pasos:

Primero, definir un conjunto de **actores**(distintas personas o dispositivos). Hay actores principales(trabajan con el software de forma directa) y secundarios(dan apoyo al sistema).

Luego, desarrollar casos de uso, se sigiere que un caso de uso responda las siguientes preguntas:

- ¿Quién es el actor principal y quién(es) el(los) secundario(s)?
- ¿Cuáles son los objetivos de los actores?
- ¿Qué precondiciones deben existir antes de comenzar la historia?
- ¿Qué tareas o funciones principales son realizadas por el actor?
- ¿Qué excepciones deben considerarse al describir la historia?
- ¿Cuáles variaciones son posibles en la interacción del actor?
- ¿Qué información del sistema adquiere, produce o cambia el actor?
- ¿Tendrá que informar el actor al sistema acerca de cambios en el ambiente externo?
- ¿Qué información desea obtener el actor del sistema?
- ¿Quiere el actor ser informado sobre cambios inesperados?

  Nota: Actor y usuario final no necesariamente son lo mismo.

Elaboracion del modelo de los requerimientos

El objetivo del modelo de analisis es describir dominios de informacion, funcion y comportamiento que se requieren. A medida que el modelo evoluciona, hay elementos que dan un fundamento solido(no cambian) y otros son volatiles, es decir, lo que indica que todavia no se entienden bien los requerimientos del sistema. Para construirlo se requieren:

- Elementos del modelo de requerimiento, algunos elementos comunes son:
  - Basados en el escenario.
  - Basados en clases.
  - De comportamiento.
  - Orientados al flujo
- Patrones de analisis

Requerimientos de las negociaciones

Generalmente, para llegar a un acuerdo con respecto a los requerimientos, se tiene que entrar en negociaciones con uno o varios participantes. Las negociaciones tienen como objetivo desarrollar un plan del proyecto que satisfalga a todos. Buscan un resiltado *win-win*. Se definen un conjunto de actividades de negociacion al principio de cada proceso de software:

- Identificación de los participantes clave del sistema o subsistema.

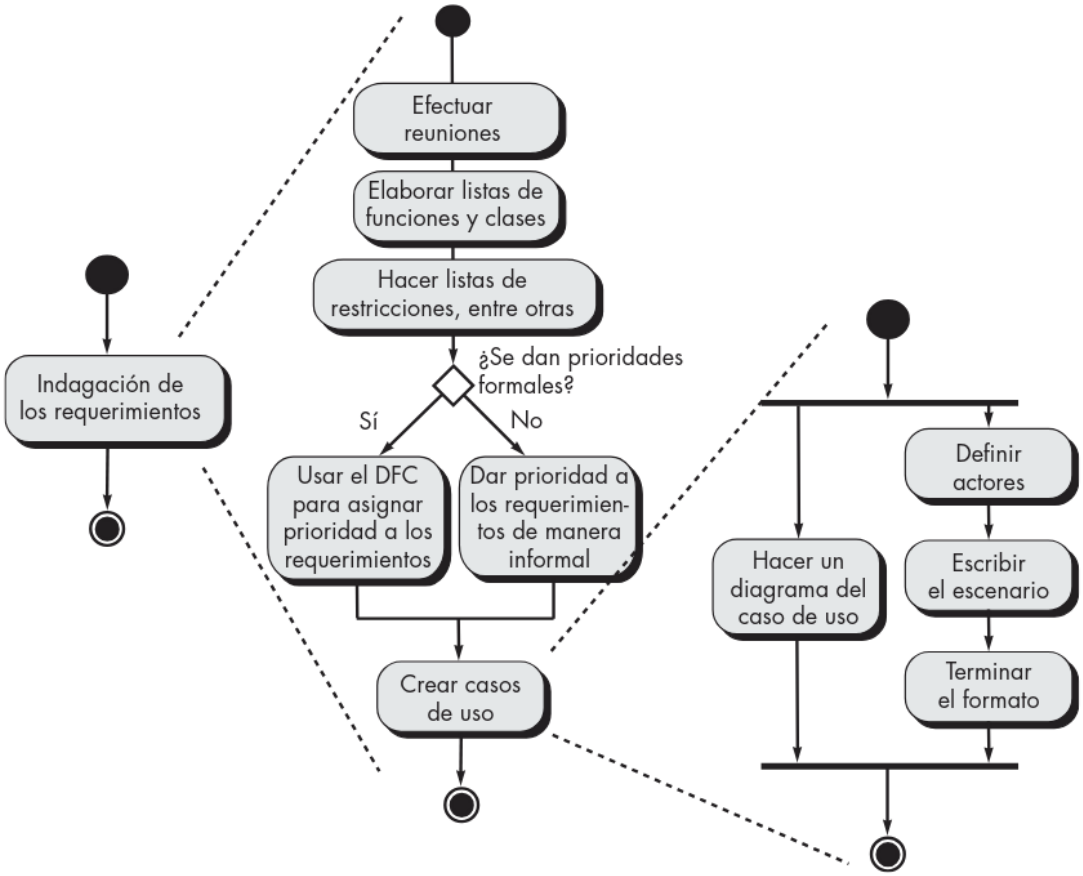

Figure 5: Diagrama UNL para indagar requerimientos

- Determinación de las “condiciones para ganar” de los participantes.
- Negociación de las condiciones para ganar de los participantes a fin de reconciliarlas en un conjunto de condiciones ganar-ganar para todos los que intervienen (incluso el equipo de software).

Validacion de los requerimientos

La revision del modelo de requerimientos para detectar inconsistencias o errores aborda las preguntas siguientes:

- ¿Es coherente cada requerimiento con los objetivos generales del sistema o producto?
- ¿Se han especificado todos los requerimientos en el nivel apropiado de abstracción? Es decir, ¿algunos de ellos tienen un nivel de detalle técnico que resulta inapropiado en esta etapa?
- El requerimiento, ¿es realmente necesario o representa una característica agregada que tal vez no sea esencial para el objetivo del sistema?
- ¿Cada requerimiento está acotado y no es ambiguo?
- ¿Tiene atribución cada requerimiento? Es decir, ¿hay una fuente (por lo general una individual y específica) clara para cada requerimiento?
- ¿Hay requerimientos en conflicto con otros? ¿Cada requerimiento es asequible en el ambiente técnico que albergará el sistema o producto?
- Una vez implementado cada requerimiento, ¿puede someterse a prueba?
- El modelo de requerimientos, ¿refleja de manera apropiada la información, la función y el comportamiento del sistema que se va a construir?
- ¿Se ha “particionado” el modelo de requerimientos en forma que exponga información cada vez más detallada sobre el sistema?
- ¿Se ha usado el patrón de requerimientos para simplificar el modelo de éstos? ¿Se han validado todos los patrones de manera apropiada? ¿Son consistentes todos los patrones con los requerimientos del cliente?

Estas deben plantearse y responderse para garantizar que el modelo refleja las necesidades del participante y provee un fundamento solido.

TL;DR

Las tareas de ingenieria de requerimientos se realizan para establecer un fundamento solido, los miembros del equipo llevan a cabo siete funciones de ingenieria de requerimientos y al final se negocia la prioridad de cada requerimientos, verificandoce que se haya entendido bien al cliente.

Capitulo 6: Modelado de los requerimientos: Escenarios, informacion y clases de analisis

El modelo de requerimientos, o conjunto de modelos, es la primera representacion tecnica de un sistema.

Analisis de los requerimientos

Da como resultado la especificacion de las caracteristicas operativas del software, permite al profesional construir sobre los requerimientos basicos establecidos durante el proceso de ingenieria de requermientos.

Esto da como resultado uno o mas tipos de modelo:

- Modelos basados en el escenario.
- Modelos de datos.
- Modelos orientados a clases.
- Modelos orientados al flujo.
- Modelos de comportamiento.

El modelo brinda al desarrollador y al cliente medios para evaluar la calidad del software final. A continuacion se centrara en el **modelado basado en escenarios**, tecnica cada ves mas popular.

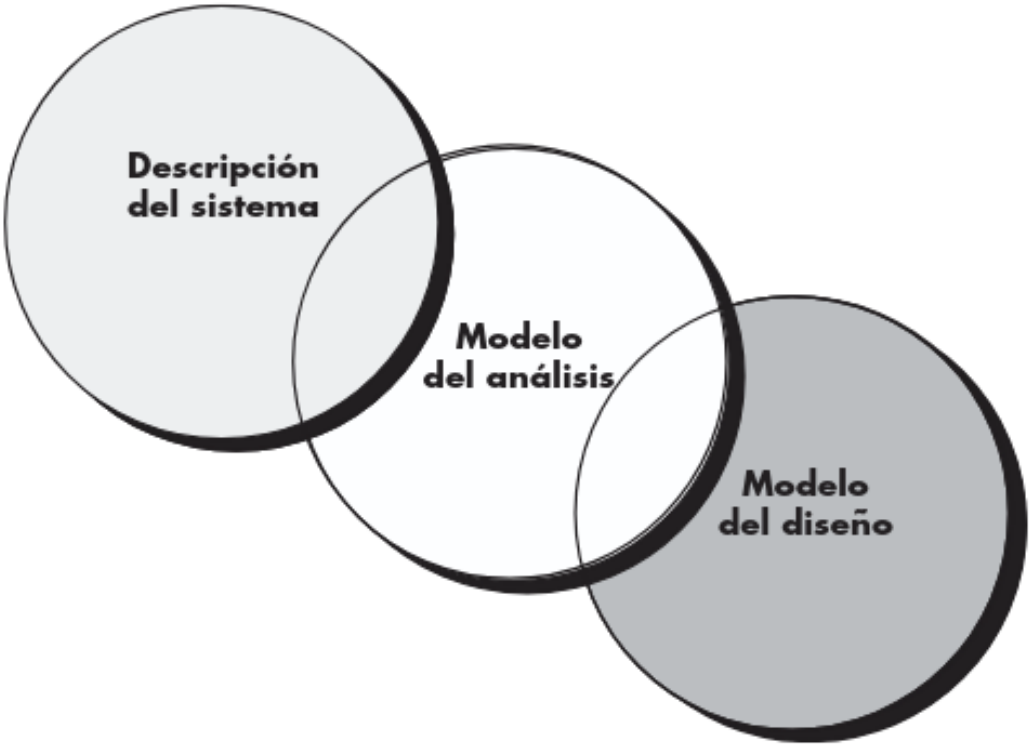

Figure 6: Modelo de requerimientos como punte

Objetivos y filosofia general

Durante el modelado se enfoca en el *que* y no en el *como*, El modelo de requerimiento debe lograr:

- Describir lo que quiere el cliente.
- Establecer una base para la creacion de un diseno de software.
- Definir un conjunto de requerimientos que puedan validarse una vez construido el software.

Ademas, todos los elementos del modelo pueden rastrearse directamente hasta las partes del modelo de diseno, diseno y analisis no estan completamente divididas(se mezclan un poco).

Reglas practicas del analisis

Se sugiere que un cierto numero de reglas practicas utiles cuando se crea un modelo:

- El modelo debe centrarse que los requerimientos visibles(no irse mucho a los detalles).
- Cada elemento del modelo debe agregarse al entendimiento general de los requermientos.
- Hay que retrasar las consideraciones de la infraestructura y otros modelos no funcionales hasta llegar a la etapa del diseno.
- Debe minimizarse el acoplamiento a traves del sistema.
- Es seguro que el modelo agrega valor para todos los participantes.
- Mantener el modelo tan sencillo como se pueda.

Analisis del dominio

Como ya se dijo, es frecuente que los patrones de analisis se repitan en muchas aplicaciones dentro del mismo negocio. Si se definen y se clasifica de manera que puedan reconocerse, la creacion del modelo sera mas expedita, y mejora el tiempo para llegar al mercado(reduciendo costos).

Se podria considerar que el analisis de dominio es una actividad sombrilla para el proceso de software, es decir, no esta conectada con ningun proyecto en particular.

Enfoques del modelado de requerimientos

- **Analisis estructurado**: Los datos y los procesos que los transforman son entidades separadas.
- **Analisis orientado a objetos**: Se centra en la definicion de las clases y en la manera en la que colaboran uno con el otro para cumplir los requerimientos. Usa UML.

Solo deben usarse elementos de modelado que agreguen valor al modelo.

Modelado basado en escenarios

La experiencia del usuario es la principal prioridad, el modelado con UML comenza con la creacion de escenarios en forma de casos de uso.

Creacion de un caso preliminar de uso

Se trata de un contrato para el comportamiento.

Para saber sobre que escribir hay que usar los dos primeras tareas de ingeniaria de requerimientos: **concepcion** e **idagacion**. Para:

- Identificar participantes
- Definir alcance
- Especificar objetivos
- Especificar prioridades
- Delinear requerimientos
- Describir objetos que seran manipulados por el sistema

Se utiliza:

- Reuniones para recabar requerimientos
- DEC

- Otros mecanismos

  Nota: Se denominan escenarios primerios aquellos que fluyen de manera lineal, sin caminos alternativos.

Mejora de un caso de uso preliminar

Se evalua cada paso del **escenario primario**, planteando:

- El actor puede emprender otra accion en este punto?
- Es posible que el actor encuentre alguna condicion de error en este punto?
- Es posible que el actor encuentre otro comportamiento?(por ejemplo, provocado por algo fuera del control del actor)

Se recomienda el uso de una sesion de "lluvia de ideas" para obtener un conjunto complejo de excepciones, luego deben plantearse:

- Existen casos en los que ocurra alguna funcion de validacion durante este caso de uso?
- Hay casos en los que una funcion(o actor) de soporte falle en responder de manera apropiada?
- El mal desempeno del sistema da como resultado acciones inesperadas o impropias?

Luego, se debe razonar una excepcion usando los siguientes criterios:

- Una excepcion debe describirse dentro del caso de uso si el software la detecta
- El software debe manejarla una vez sea detectada.

Modelos UML que proporcionan el caso de uso

Muchas veces un modelo basado en texto es muy poco util, entonces, se utilizan modelos UML graficos. Hay varios tipos de diagramas:

- Diagrama de actividades
- Diagramas canal

Concepto de modelado de datos

Si el software debe trabajar con datos, el equipo de software muy probablemente haga un **modelo de datos** como parte del modelado de requerimientos. Para eso, se utilizan los diagramas **entidad-relacion**.

(Interseccion con Base de Datos 1, ver apuntes esa materia)

Modelado basado en clases

Representa:

- Los objetos que manipularan el sistema
- Las operaciones que se aplicaran a los objetos
- Las relaciones entre objetos
- Colaboraciones que ttienen lugar entre clases

Los elementos de este tipo de modelo son:

- Clases

- Objetos
- Atributos
- Paquetes

Identificacion de las clases de analisis

Se manifiestan en uno de los modos siguientes:

- **Entidades externas** (por ejemplo, otros sistemas, dispositivos y personas) que producen o consumen la información que usará un sistema basado en computadora.
- **Cosa**s (reportes, pantallas, cartas, señales, etc.) que forman parte del dominio de infor- mación para el problema.
- **Ocurrencias o eventos** (como una transferencia de propiedad o la ejecución de una serie de movimientos de un robot) que suceden dentro del contexto de la operación del sistema.
- **Roles** (gerente, ingeniero, vendedor, etc.) que desempeñan las personas que interactúan con el sistema.
- **Unidades organizacionales** (división, grupo, equipo, etc.) que son relevantes para una aplicación.
- **Lugares** (piso de manufactura o plataforma de carga) que establecen el contexto del problema y la función general del sistema.
- **Estructuras** (sensores, vehículos de cuatro ruedas, computadoras, etc.) que definen una clase de objetos o clases relacionadas de éstos.

Esta es una de las muchas propuestas. Es importante darse cuenta de lo que no son las clases u objetos, una clase nunca debe tener un "nombre de procedimiento imperativo".

Especificacion de atributos

Describen una clase que fue seleccionada para incluirse en el modelo de requerimientos.

Definicion de las operaciones

Definen el comportamiento de un objeto, se dividen en cuatro categorias:

- Operaciones quemanipulan datos en cierta manera (por ejemplo, los agregan, eliminan, editan, seleccionan, etc.)
- Operaciones que realizan un cálculo.
- Operaciones que preguntan sobre el estado de un objeto.
- Operaciones que vigilan un objeto en cuanto a la ocurrencia de un evento de control.

Modelado clase-responsabilidad-colaborador(CRC)

Proporciona una manera sencilla de identificación y organización de las clases que son relevantes para los requerimientos de un sistema o producto.

Las clases mencionadas anteriormente se pueden categorizar de las siguientes maneras:

- Clases de entidad, se extraen directamente del enunciado del problema.
- Clases de frontera, para crear la interfaz.
- Clases de controlador, administran una unidad de trabajo de principio a fin.

Se sugieren los siguientes lineamientos para asignar responsabilidades:

- La inteligencia del sistema debe estar distribuida entre las clases para enfrentar mejor las necesidades del problema.
- Cada responsabilidad debe enunciarse del modo más general posible.
- La información y el comportamiento relacionado con ella deben residir dentro de la misma clase.
- La información sobre una cosa debe localizarse con una sola clase, y no distribuirse a través de muchas.
- Cuando sea apropiado, las responsabilidades deben compartirse entre clases relacionadas.

Una de las maneras en la que una clase cumple sus responsabilidades es colaborando. Las colaboraciones se identifican determinando si una clase puede cumplir cada responsabilidad. Si no es así, entonces necesita interactuar con otra clase. Ésa es una colaboración.

Cuando se desarrolla un modelo CRC completo, los participantes lo revisan de la siguiente manera:

1. Se da a todos los participantes que intervienen en la revisión (del modelo CRC) un subconjunto del modelo de tarjetas índice CRC. Deben separarse aquellas que colaboran(de modo que ningún revisor deba tener dos tarjetas que colaboren).
2. Todos los escenarios de casos de uso (y los diagramas correspondientes) deben organizarse en dos categorías.
3. El líder de la revisión lee el caso de uso en forma deliberada. Cuando llega a un objeto con nombre, entrega una ficha a la persona que tenga la tarjeta índice de la clase correspondi
4. Cuando se pasa la ficha, se pide al poseedor de la tarjeta Sensor que describa las responsabilidades anotadas en la tarjeta. El grupo determina si una (o más) de las responsabilidades satisfacen el requerimiento del caso de uso.
5. Si las responsabilidades y colaboraciones anotadas en las tarjetas índice no se acomodan al caso de uso, éstas se modifican. Lo anterior tal vez incluya la definición de nuevas clases (y las tarjetas CRC índice correspondientes) o la especificación en las tarjetas existentes de responsabilidades o colaboraciones nuevas o revisadas.

Asociaciones y dependencias

En muchos casos, dos clases de análisis se relacionan de cierto modo con otra, en forma muy parecida a como dos objetos de datos se relacionan entre sí.

Paquete de analisis

na parte importante del modelado del análisis es la categorización. Es decir, se clasifican dis- tintos elementos del modelo de análisis (por ejemplo, casos de uso y clases de análisis) de manera que se agrupen en un paquete llamado **paquete de análisis** al que se da un nombre representativo.

TL:DR

El objetivo del modelado de los requerimientos es crear varias representaciones que describan lo que necesita el cliente, establecer una base para generar un diseño de software y definir un conjunto de requerimientos que puedan ser validados una vez construido el software. El modelo de requerimientos cruza la brecha entre la representación del sistema que describe el sistema en su conjunto y la funcionalidad del negocio, y un diseño de software que describe la arquitectura de la aplicación del software, la interfaz de usuario y la estructura de componentes.

Los modelos basados en el escenario ilustran los requerimientos del software desde el punto de vista del

usuario. El modelado basado en clases utiliza información obtenida de los elementos del modelado basado en el escenario y en datos, para identificar las clases de análisis